Where Should a Robot Swarm See Its Neighbors? Communication-Range Transfer in a GNN Coverage Policy

Loïc Poisson **Harold Gardon** **Claire Fitussi**
Stanford University
{lpoisson, gardon21, cfitussi}@stanford.edu

Status in one paragraph. Since the proposal we have (i) finished the diagnosis of LPAC's failure when the communication range grows, (ii) implemented and trained six variants of the policy (three seeds each, 25 training runs on a FarmShare L40S GPU), and (iii) found the first trained model whose coverage improves when robots hear farther: a GNN layer that reads *bounded* relative geometry on the edges (V4). We also learned two methodological lessons that change how we will report results. This draft is organized as the final blog post will be: motivation, data, model, results, and what remains.

1 Problem and motivation

In *coverage control*, N robots spread over an area so that their sensors watch the regions that matter, given by an importance density field (IDF) ϕ unknown in advance [Cortés et al., 2004]. Each robot sees only a 64×64 m footprint and talks only to robots within its *communication range* R. The communication network is the graph $G_R = (V, E_R)$, with an edge (i, j) whenever $\|p_i - p_j\| \leq R$, so decentralized control is a graph learning problem: each robot decides what to send to its neighbors and how to combine what it receives [Tolstaya et al., 2020].

LPAC [Agarwal et al., 2025] is a recent GNN policy for this task. Its authors report that it degrades when R grows (their Sec. VI-H, Fig. 20) and hypothesize that aggregating many messages loses information. This matters in practice: a radio's range changes with terrain and hardware, and a fleet may mix robots with different radios. A policy that gets *worse* when robots communicate *more* cannot be deployed safely. Our question is therefore: *where should neighbor geometry enter a GNN policy, and at which scale, so that the policy transfers across communication ranges and benefits from longer ones?*

2 Data, processing and metric

Dataset. We use the public **CoverageControl** library [KumarRobotics, 2024] (GPL-3.0, release v1.3.1): a simulator of 32 robots in a 1024×1024 m world whose IDFs are mixtures of 32 random Gaussians, the published LPAC checkpoint, its 11 evaluation environments, and scripts that generate imitation data from a *clairvoyant* centroidal-Voronoi (CVT) controller that knows ϕ and every position.

Processing. One sample is the state of all 32 robots at one time step: four 32×32 maps per robot (local importance, two neighbor-map channels, obstacles), the positions, the graph G_R and the clairvoyant velocity as target. As in the paper, a training set has 100,000 samples. We generated sets at 128, 256, 512 and 1024 m, merged them into a *mixed-range* set, and wrote a redrawing tool that rebuilds the neighbor maps of an existing set at any scale (it recovers each sample's range from its stored edge weights), so a new map encoding does not require re-simulating. We also generated 40 new evaluation environments.

CS224W: Machine Learning with Graphs, Stanford University, Autumn 2026. Project milestone (type 1: real-world application).

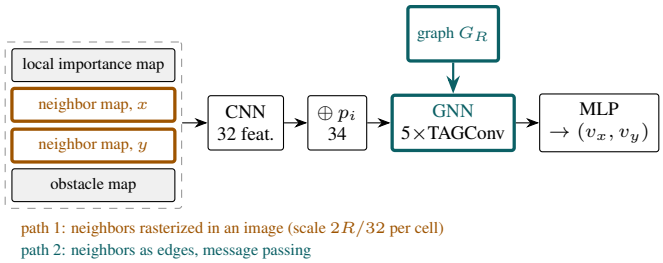


Figure 1: LPAC on one robot. Neighbors enter twice: drawn in a 32×32 neighbor map read by the CNN (path 1), and as edges of G_R read by the GNN (path 2).

Task and metric. The policy maps local observations and messages to a velocity (node-level regression on G_R), trained by imitation with a mean-squared error. We evaluate it in closed loop for 600 steps with the paper's metric: the coverage cost $J = \int \phi(q) \min_i \|q - p_i\|^2 \, dq$ averaged over the episode and divided by the clairvoyant's cost on the same environment (lower is better). We vary the *test-time* range $R \in \{128, 256, 512, 768, 1024\}$ m. Baselines are the centralized CVT (about 2.3–2.5) and the published model.

Protocol and noise. Evaluation is deterministic on Intel CPU nodes (AMD nodes round convolutions differently). Differences between models are paired by environment. We report the 40 new environments as the primary set: two runs of the same recipe differ there by 0.01–0.05, against 0.08–0.15 on the 11 released ones (Sec. 4).

3 Model design

LPAC. Each robot runs the same network (Fig. 1). A CNN encodes the four maps into 32 features, the position is appended, five TAGConv layers [Du et al., 2017] with $K = 3$ exchange information over G_R, and an MLP outputs the velocity:

$$X^{(\ell+1)} = \mathrm{ReLU}\Big(\textstyle\sum_{k=0}^{K} \big(D^{-1/2} A D^{-1/2}\big)^k X^{(\ell)} W_k^{(\ell)} \Big). \tag{1}$$

In the *neighbor map*, robot j within range is written into cell $c_{ij} = \mathrm{round}\big((p_j - p_i)\frac{32}{2R} + 15.5\big)$ with value $(p_j - p_i)/R$. The map therefore zooms out with R: a cell is 8 m wide at 128 m and 64 m at 1024 m, and a model trained at one range sees images at test time that it never saw in training.

Variants. All variants keep the TAGConv stack and the training recipe (imitation, Adam, best validation checkpoint) and change only how neighbor geometry is encoded. *Mixed* means trained on the merged 128–1024 m set.

- **A**: LPAC with the neighbor map drawn at a fixed 256 m scale (cells of 16 m, neighbors within $\min(R, 256)$ m).
- **V1**: no neighbor map. **V2**: GATv2 [Brody et al., 2022] with $p_j - p_i$ as edge feature instead of TAGConv.
- **V3**: V1 plus a residual PointNet layer [Qi et al., 2017] before the TAGConv stack, with edge input $(p_j - p_i)/256$ m.
- **V4** (bounded edges): V3 with an edge input that stays in $[-1, 1]$ at every range. With $d_{ij} = \|p_j - p_i\|$,

$$e_{ij} = \Big[\tfrac{p_j - p_i}{d_{ij}},\ \tanh\big(\tfrac{d_{ij}}{256}\big) \Big], \qquad h_i = x_i + \mathrm{ReLU}\Big(g\big(\max_{j \in \mathcal{N}(i) \cup \{i\}} f([x_j, e_{ij}]) \big) \Big), \tag{2}$$

  with f, g two-layer MLPs, max aggregation (invariant to the number of neighbors, about 2 at 128 m and 30 at 1024 m) and $e_{ii} = 0$ for the self-loop.
- **V5** (multi-scale maps): LPAC with three neighbor-map pairs at fixed scales $s \in \{256, 512, 1024\}$ m, each holding the neighbors within $\min(R, s)$, so the CNN reads 8 channels.
- **V6**: V5 plus the V4 edge layer.

Table 1: Cost / clairvoyant cost (lower is better), 40 new environments, mean over three training seeds (six runs for V5). The inference-time fix is shown for reference; all other rows are trained models used as is.

Model	128 m	256 m	512 m	768 m	1024 m
LPAC published (trained 128 m)	1.96	1.70	2.87	–	6.15
+ fixed 256 m map at inference	–	1.70	1.71	–	1.77
Centralized CVT	2.33	2.33	2.33	–	2.33
A: fixed map, mixed	2.09	1.91	1.91	1.91	1.92
V3: PointNet edges, trained 128 m	2.13	2.03	2.25	2.39	2.44
V4: bounded edges, trained 128 m	2.05	1.87	1.90	1.90	1.90
V4: bounded edges, mixed	2.14	1.87	**1.81**	**1.81**	**1.82**
V5: multi-scale maps, mixed	2.09	1.89	1.89	1.90	1.90
V6: V5 + V4, mixed	2.12	1.91	1.87	1.88	1.89

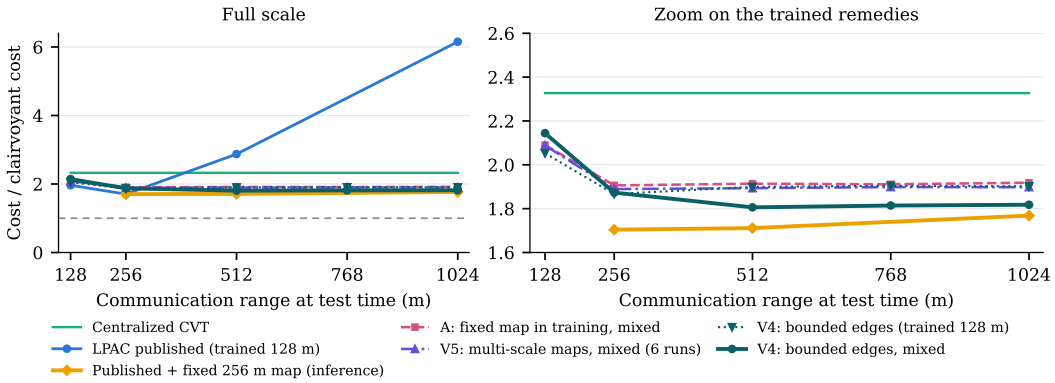


Figure 2: Cost against test-time range on the 40 new environments. Left: the published model collapses. Right: every trained remedy removes the collapse; only V4 trained on mixed ranges gets better beyond 256 m.

4 Results so far

1. The collapse comes from the image, not from the graph. Retrained, LPAC matches the published model (1.74 vs 1.69 at 256 m). Trained at 128 m, it reaches 5.22 at 1024 m (6.15 on the 40 new environments), worse than the centralized CVT. A factorial intervention at inference separates the two paths. Capping G_R to the 4 nearest neighbors leaves the cost at 5.11. Redrawing only the neighbor map at a fixed 256 m scale, with the full graph, brings it to 1.71 (paired $+0.02 \pm 0.16$ against 256 m). This holds for every model and on the 40 new environments, and the effective rank of TAGConv outputs never collapses, which rules out over-smoothing. The paper's hypothesis, a lossy aggregation of many messages, is therefore not the cause in transfer.

2. Bounded geometry on the edges removes the collapse. Table 1 and Fig. 2 compare the trained remedies. Removing the map (V1) costs 0.75 at 256 m, and GATv2 on raw relative positions (V2) underfits. V3's PointNet layer degrades with range because its inputs grow with R. V4 replaces them with a unit direction and a saturating distance: *trained at 128 m only*, it no longer collapses (1.90 at 1024 m, three seeds) and beats V3 at every range, beyond seed noise.

3. A first model that benefits from a longer range. Trained on mixed ranges, V4 is the only model whose cost *decreases* from 256 to 1024 m: -0.055 (SE 0.016, 27/40 environments better) on the new environments and -0.12 (SE 0.03, 10/11) on the released ones, in each of the three seeds. A and V5 are flat, which means they tolerate a longer range without using it. V6 behaves exactly like V5 (within 0.03 at every range), so the two encodings do not add up. No trained model yet reaches LPAC's 256 m level: at 256 m all stay 0.05–0.09 behind LPAC retrained at that range (1.82 on the new environments).

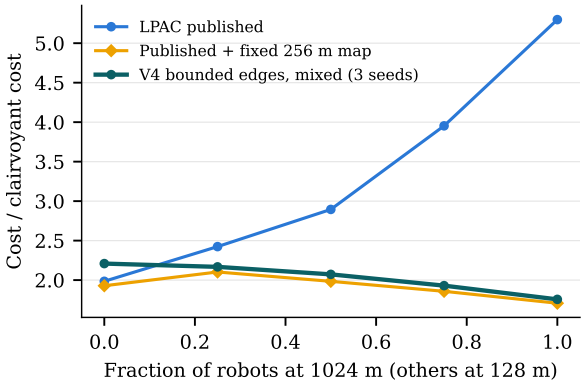


Figure 3: Heterogeneous teams on the 11 released environments: a fraction of the robots hears at 1024 m, the others at 128 m (8 random draws for LPAC, 4 per seed for V4).

4. Heterogeneous teams. When a fraction f of the robots hears at 1024 m and the rest at 128 m (Fig. 3), the published model's cost grows from 1.98 to 5.30. With V4 (mixed), each robot that hears farther helps: the cost falls at every step of f for each seed, by 0.45 in total (SE 0.08, 11/11 environments). This matches the inference-time fix within noise ($+0.05$ to $+0.09$, SE 0.08) for $f \geq \frac{1}{4}$, without any test-time change. V4 remains 0.28 behind when every robot is at 128 m, its weakest range.

5. Two methodological lessons. *(a) Run-to-run noise.* Training is not bit-reproducible at a fixed seed (GPU scatter operations, parallel data loading). Two launches of V5 with the same seed differ by 0.08–0.15 at 256 m on the 11 released environments, more than the spread between seeds of one launch. A first set of three V5 runs appeared to match LPAC at 256 m (1.78); pooled with three more, it is 0.10 behind. We now report the 40 new environments as primary and will use six or more runs per recipe for any claim at the 0.05 level. *(b) Validation loss does not predict coverage.* Map-based models reach their best validation MSE at epoch 4–6, after which it rises from 0.46 to 0.54 while the coverage cost does not move: epoch-25 checkpoints of V5 and V6 match their best-validation checkpoints within 0.04. Imitation loss is therefore a poor model-selection signal for this task.

5 Code

The repository[1] contains: `lpac_study/`, a Python package around CoverageControl and PyTorch Geometric [Fey and Lenssen, 2019] with the closed-loop evaluation (`episode.py`, `sweep.py`), the inference-time interventions on the graph and the map (`factorial.py`, `neighbourhood.py`), the variants V0–V6 (`variants.py`), heterogeneous teams (`heterogeneous.py`), the specialist mixture (`mixture.py`) and the analyses (effective rank, CKA, checkpoint selection); `training/`, data generation, map redrawing and the Slurm launchers for FarmShare; `tests/`, unit tests of the variants (bounds and scale behavior of the V4 features, equality of V5's 256 m pair with the fixed map, V6 reducing to V5 when its edge layer is zeroed); the result tables and a running journal of every experiment. An early notebook (`lpac_eval.ipynb`) runs the evaluation of the published model; it will grow into the Colab.

6 Plan to the final submission

1. **Firm up the main claims**: six runs for LPAC retrained at 256 m, A, V4 mixed and V5 mixed, so that the 256 m gap and the range gain are measured against run-to-run noise.
2. **Close V4's gap at short range**: V4 is weak at 128 m (2.14). We will try weighting the mixed set toward short ranges and a learned scale in the saturating distance.

[1] https://github.com/loicpoisson/lpac-communication-range (private until the final submission; access on request).

3. **Information loss vs. distribution shift**: draw the same neighbors in a finer map to test whether the 256 m map discards useful information.
4. **Deliverables**: the Colab (data loading, model definitions in PyG, training on a reduced set, closed-loop evaluation of the released checkpoints) and the Medium post built on Figs. 1–3.

References

Saurav Agarwal, Ramya Muthukrishnan, Walker Gosrich, Vijay Kumar, and Alejandro Ribeiro. LPAC: Learnable perception-action-communication loops with applications to coverage control. *IEEE Transactions on Robotics*, 41, 2025. arXiv:2401.04855.

Shaked Brody, Uri Alon, and Eran Yahav. How attentive are graph attention networks? In *International Conference on Learning Representations*, 2022.

Jorge Cortés, Sonia Martínez, Timur Karataş, and Francesco Bullo. Coverage control for mobile sensing networks. *IEEE Transactions on Robotics and Automation*, 20(2):243–255, 2004.

Jian Du, Shanghang Zhang, Guanhang Wu, José M. F. Moura, and Soummya Kar. Topology adaptive graph convolutional networks. *arXiv preprint arXiv:1710.10370*, 2017.

Matthias Fey and Jan Eric Lenssen. Fast graph representation learning with PyTorch Geometric. In *ICLR Workshop on Representation Learning on Graphs and Manifolds*, 2019.

KumarRobotics. CoverageControl: A library for coverage control with robot swarms, release v1.3.1. `https://github.com/KumarRobotics/CoverageControl`, 2024.

Charles R. Qi, Li Yi, Hao Su, and Leonidas J. Guibas. PointNet++: Deep hierarchical feature learning on point sets in a metric space. In *Advances in Neural Information Processing Systems*, 2017.

Ekaterina Tolstaya, Fernando Gama, James Paulos, George Pappas, Vijay Kumar, and Alejandro Ribeiro. Learning decentralized controllers for robot swarms with graph neural networks. In *Conference on Robot Learning*, volume 100 of *Proceedings of Machine Learning Research*, pages 671–682, 2020.